

Resumo executivo: modelo v4 original vs nova versao (13/02/2026)

Objetivo deste documento

Este resumo compara:

- como estava o modelo v4 (antes dos ajustes recentes),
- como esta a nova versao em producao,
- e o que recomendamos para a proxima versao (v5).

1) Como estava o modelo v4 (base original)

Arquitetura operacional

- Detector H3B em tempo real via WebSocket.
- Quando havia sinal, a auditoria API consultava o betslip para capturar preco real.
- Captura simultanea de Back e Lay no instante T+0.
- Curva temporal focada principalmente no Back.

O que funcionava bem

- Pipeline rapido para auditoria sem DOM completo.
- Captura de odds Back consistente.
- Persistencia de auditorias H3B em `betslip\_audit\_results`.

Limites identificados

- **Lay temporal nao era equivalente ao Back** (faltava serie temporal dedicada de Lay).
- **Telemetria incompleta de ponta a ponta**: havia lag total e alguns tempos intermediarios, mas sem trilha completa por etapa para diagnostico fino de fila/gargalo.
- Visibilidade operacional dispersa (sem JSONL padronizado para telemetria de auditoria e coletor).

2) Nova versao (estado atual validado em producao)

Melhorias implementadas

- Lay temporal equivalente ao Back nos checkpoints:
 - `t+3, t+6, t+10, t+15, t+20`.
- Persistencia em `hypothesis\_details` de:
 - `lay` (snapshot T+0),
 - `temporal` (Back temporal),
 - `lay\_temporal` (Lay temporal),
 - `telemetry` (tempos detalhados).
- Telemetria por auditoria com etapas:
 - fila, build, fetch paralelo, post/pmm back+lay, temporal, db, total e overhead.
- Telemetria do coletor por ciclo:

- `collect\_ms`, `save\_ms`, `cycle\_total\_ms`, payload/saved, pre/live, erros.
- Healthcheck one-shot com PASS/WARN/FAIL:
- `healthcheck\_v4.sh`.

Evidencias recentes (13/02/2026)

- Serviços ativos (`collector` e `audit-api`).
- Arquivos JSONL de telemetria presentes:
- `logs/audit\_api\_telemetry.jsonl`
- `logs/collector\_telemetry.jsonl`
- Banco com registros novos contendo telemetria e lay temporal:
- Exemplo recente: linha com `has\_telemetry = true`, `has\_back\_temporal = true`, `has\_lay\_temporal = true`.

Leitura de transicao

- Parte do historico antigo continua sem `telemetry` e sem `lay\_temporal` (normal, pois sao linhas anteriores ao corte de versao).
- Ocorrencias de `API\_FAILED` existem em periodos de instabilidade de proxy/tunel.

3) Comparativo direto (v4 original x nova versao)

Dimensao	v4 original	Nova versao
Captura Back T+0	Sim	Sim
Captura Lay T+0	Sim	Sim
Curva temporal Back	Parcial/ativa	Ativa e mantida
Curva temporal Lay	Nao equivalente ao Back	Equivalente ao Back (t+3/6/10/15/20)
Telemetria ponta a ponta	Limitada	Completa por etapa + overhead
Telemetria operacional em arquivo	Nao padronizada	JSONL no audit e no collector
Validacao rapida operacional	Manual/dispersa	`healthcheck_v4.sh`
Diagnostico de fila/gargalo	Baixa granularidade	Alta granularidade

4) Pontos ainda sensiveis

- **Confiabilidade de proxy:** ainda ha episodios de `ERR\_TUNNEL\_CONNECTION\_FAILED`/timeout em alguns intervalos.
- **Cobertura estatistica:** lay temporal ja existe, mas ainda precisa aumentar N para conclusoes mais robustas por bucket.
- **Dados legados:** linhas anteriores ao corte nao possuem telemetria nova.

5) Sugestoes para a proxima versao (v5)

Prioridade alta (impacto imediato)

- **V5.1 Telemetria first-class**

- Criar tabela dedicada de telemetria (alem do JSON) para consultas rapidas, alertas e dashboards.
- Campos minimos: `queued\_ms`, `exec\_ms`, `api\_ms`, `temporal\_ms`, `db\_ms`, `pipeline\_ms`, `proxy\_status`.

- **V5.2 Resiliencia de proxy/rede**

- Healthcheck ativo do proxy antes do ciclo.
- Retry com backoff e circuit breaker.
- Fallback controlado (quando permitido) para reduzir downtime.

- **V5.3 Temporal desacoplado**

- Separar auditoria T+0 (critica) do worker temporal (t+3/6/10/15/20), para reduzir impacto de bloqueio no fluxo principal.

Prioridade media

- **V5.4 Modelo de dados evolutivo**

- Migrar `hypothesis\_details` para `jsonb` nativo e criar indexes funcionais.
- Materializar campos-chave (ex.: `has\_lay\_temporal`, `pipeline\_total\_ms`) para leitura analitica.

- **V5.5 Backfill historico automatizado**

- Job idempotente para preencher telemetria/lay temporal quando houver evidencias em log.

Prioridade estrategica

- **V5.6 Motor de decisao Lay por contexto**

- Threshold dinamico por regime (pre/in), lag e bucket BS vs WS.
- Meta: aumentar N util sem degradar qualidade de execucao.

6) Plano sugerido de entrega (enxuto)

Fase	Janela	Entrega
Fase 1	3-5 dias	Hardening de proxy + alertas + estabilidade de servico
Fase 2	5-7 dias	Tabela de telemetria + dashboard + SLO operacional
Fase 3	7-10 dias	Motor de decisao Lay contextual + avaliacao de impacto em N e qualidade

7) Conclusao executiva

- A nova versao representa um salto relevante em observabilidade e qualidade de captura (especialmente para Lay temporal).
- A base ja suporta analise de etapas intermediarias e ponta a ponta, o que era requisito central.

- O principal risco remanescente para performance real e estabilidade de rede/proxy; tratar isso na v5 deve ser prioridade numero 1.